

- Business Requirements Document (BRD)**
- Project Title:** Data Integration and Reporting for Customer and Order Data
- Introduction:**

The purpose of this project is to design and implement a data integration and reporting solution that combines customer and order data from various sources into a unified view, enabling business stakeholders to analyze customer spending patterns and make informed decisions.

- Business Requirements:**

1. **Data Ingestion**

- Read source CSV data from a specified volume location into Delta tables for customer and order data.
- The CSV files are located at ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata`` and ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata``.

2. **Data Quality**

- Remove "Null"/Null and duplicate records from both customer and order tables.

3. **Data Transformation and Loading**

- Create an "ordersummary" table if it does not exist in the specified catalog (``gen_ai_poc_databrickscoe``) and schema (``sdlc_wizard``).
- Join customer and order data using the "CustId" field and load the data into an SCD type 2 table (``ordersummary``).
- The schema of the "ordersummary" table is: ``CustId``, ``Name``, ``EmailId``, ``Region``, ``OrderId``, ``ItemName``, ``PricePerUnit``, ``Qty``, ``Date``.

4. **SCD Type 2 Implementation**

- Implement SCD type 2 logic to update the "ordersummary" table whenever there is a change in the customer table.
- Old records should be made inactive, and new records should be made active.
- Update ``StartDate`` and ``EndDate`` accordingly.

5. **Aggregated Reporting**

- Create a new table "customeraggregatespend" with columns "Name", "TotalAmount", and "Date" in the specified catalog (``gen_ai_poc_databrickscoe``) and schema (``sdlc_wizard``) if it does not exist.
- Aggregate the "TotalAmount" column from the "ordersummary" table and group by "Name" and "Date" columns.
- Load the aggregated data into the "customeraggregatespend" table.

- Functional Requirements:**

1. The system should be able to read CSV data from the specified volume location.
2. The system should be able to remove "Null"/Null and duplicate records from the customer and order tables.
3. The system should be able to create the "ordersummary" table if it does not exist.
4. The system should be able to join customer and order data using the "CustId" field and load the data into the "ordersummary" table.
5. The system should implement SCD type 2 logic to update the "ordersummary" table.
6. The system should be able to create the "customeraggregatespend" table if it does not exist.
7. The system should be able to aggregate data from the "ordersummary" table and load it into the "customeraggregatespend" table.

- Non-Functional Requirements:**

1. Data quality checks should be performed to ensure data accuracy and consistency.
2. The system should be able to handle large volumes of data.
3. The system should be designed to ensure data security and integrity.

- Assumptions and Dependencies:**

1. The CSV files are in the correct format and are located at the specified volume location.
2. The necessary catalog and schema exist in the Databricks environment.
3. The necessary permissions and access rights are granted to the users.

- Success Criteria:**

1. The data is successfully ingested into the Delta tables.
2. The data quality checks are passed, and the data is accurate and consistent.
3. The "ordersummary" table is created and populated with the joined data.
4. The SCD type 2 logic is implemented correctly, and the "ordersummary" table is updated accordingly.
5. The "customeraggregatespend" table is created and populated with the aggregated data.

- Acceptance Criteria:**

1. Verify that the data is correctly ingested into the Delta tables.
2. Verify that the data quality checks are passed, and the data is accurate and consistent.

3. Verify that the "ordersummary" table is created and populated with the joined data.
4. Verify that the SCD type 2 logic is implemented correctly, and the "ordersummary" table is updated accordingly.
5. Verify that the "customeraggregatespend" table is created and populated with the aggregated data.